

Southwest  
Tech

# Python Programming Foundations

Week 6 - API's, External Data, and Responsible AI Use  
Day 1: "Sequential & Asynchronous Thinking"

1

---

---

---

---

---

---

---

---

Southwest  
Tech

## Sometimes Programs Have To Wait

-  Some programs can do everything with local values and files.
-  Other programs ask for information from somewhere else, and that outside response may take time.
-  When that happens, the program has a waiting point. Waiting does not make the program wrong; it means the program depends on a response before it can continue.

2

---

---

---

---

---

---

---

---

Southwest  
Tech

## How a program waits for and uses data



```
graph LR; A[Step 1: ask for data] --> B[waiting]; B --> C[Step 2: use response];
```

**Step 1: ask for data**  
The program asks the user (or another source) for data.

**waiting**  
The program pauses and waits for a response.

**Step 2: use response**  
The program gets the response and uses it to continue.

## Programs May Need To Wait

3

---

---

---

---

---

---

---

---



## Review: Local Data Was Available Immediately



- ▶ Last week, our data was stored locally.
- ▶ The file might be missing or invalid, but the program was still looking for information in a known place.
- ▶ External data changes that situation. The program may need to ask another system, wait for that system, and then decide whether the response is useful.

4

---

---

---

---

---

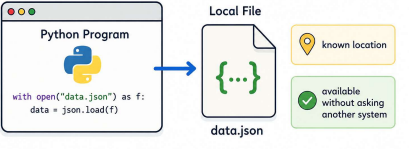
---

---

---



## Local Data Was In A Known Place



Local files can still fail, but the program knows where to look.

## Review: Local Data

5

---

---

---

---

---

---

---

---




## Review Coding Lab

6

---

---

---

---

---

---

---

---

Southwest Tech

## What Counts as Success Today

The main points:

- recognize where the program asks for outside data
- identify the waiting point
- inspect what response comes back
- select useful values from the response
- explain how the program handled the dependency

The goal is not to *memorize* async syntax.  
The goal is to *recognize* the shape of the problem.

7

---

---

---

---

---

---

---

---

Southwest Tech

## Request/Response Success Pattern

A simple way to think about getting outside data

1. ask for outside data  
Tell the outside source what you need.

2. notice the waiting point  
The program pauses here until a response comes back.

3. inspect the response  
Look at the response structure and content to understand it.

4. select useful values  
Pick the pieces of data you need for your program.

5. explain the dependency  
State clearly that your program depends on outside data.

★ Clear thinking about each step leads to more reliable and easier-to-debug programs.

## Today's Success

8

---

---

---

---

---

---

---

---

Southwest Tech

## What We Will Use Today

- sequential flow
- request
- response
- waiting point
- simulated response
- beginner-level async recognition

*These words help us describe what the program is doing before we worry about advanced syntax.*

9

---

---

---

---

---

---

---

---

Southwest  
Tech

## Today's External Data Ideas

Key ideas we will use when working with outside data.

**1 sequential flow**

Our program moves step by step in order.

**2 request**

We ask an outside source for data we need.

**3 response**

The source sends data back to us.

**4 waiting point**

Our program pauses here until the response arrives.

**5 simulated response**

We can create a pretend response to test our program.

**6 async recognition**

We recognize that some tasks happen over time.

★ These ideas help us build programs that get outside data reliably and clearly.

## Toolbox

10

---

---

---

---

---

---

---

---

Southwest  
Tech

## What We Will “Skip” For Now

- ▶ writing asynchronous programs
- ▶ `'async' / 'await'` mastery
- ▶ concurrent design
- ▶ API authentication
- ▶ deployment

*Today we recognize the idea without making it a required coding target.*

11

---

---

---

---

---

---

---

---

Southwest  
Tech

## Later, Not Today's Target

This shelf is for important ideas we will return to later.

`'async' / 'await'`

concurrency

authentication

deployment

full API implementation

Useful later, not required today.

Keep learning. Build step by step.

## Parked For Later

12

---

---

---

---

---

---

---

---



13

---

---

---

---

---

---

---

---

Southwest Tech

## Sequential Flow Happens In Order

**Sequential Flow**

Python usually completes one instruction, then moves to the next.

- ▶ Sequential code runs step by step.
- ▶ Python completes one instruction, then moves to the next instruction.
- ▶ Most of the programs you have written so far follow this pattern: start at the top, run the next line, and continue in order.

14

---

---

---

---

---

---

---

---

Southwest Tech

## Requests Interrupt The Simple Story

A request asks another system for information.

The program may need to wait before it can use the response.

A beginner-friendly pattern is:

- ▶ ask
- ▶ wait
- ▶ receive
- ▶ inspect
- ▶ use

15

---

---

---

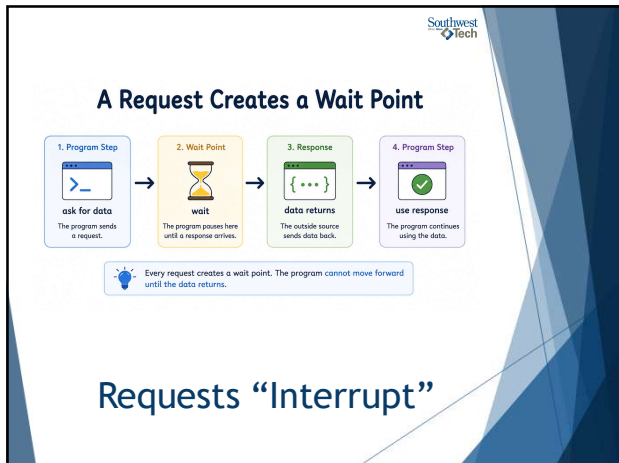
---

---

---

---

---



16

---

---

---

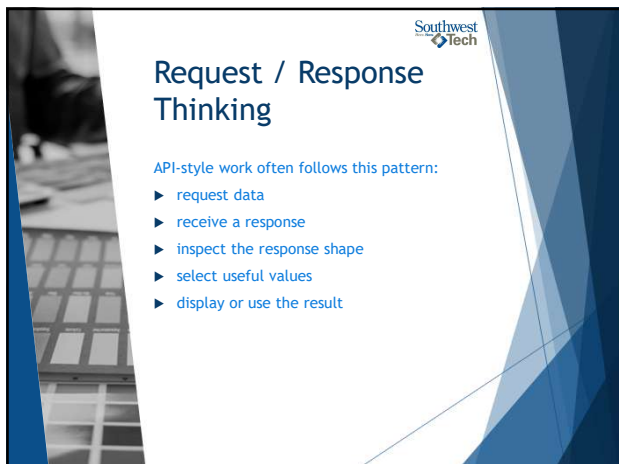
---

---

---

---

---



17

---

---

---

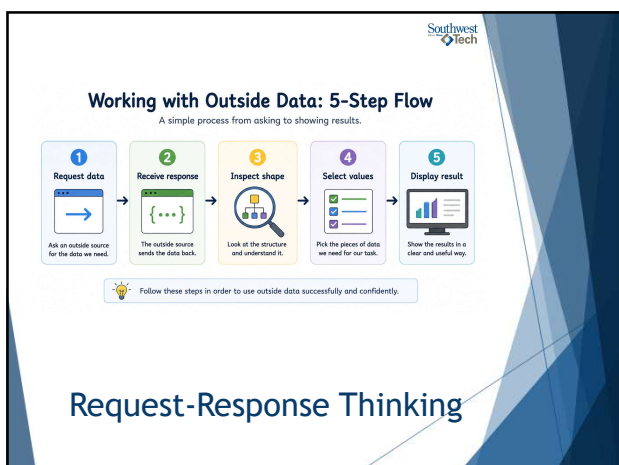
---

---

---

---

---



18

---

---

---

---

---

---

---

---



Southwest  
Tech

## Waiting Does Not Always Mean Broken

- ▶ When outside data is involved, delay can be normal.
- ▶ The real questions are: what is the program waiting for, and what should happen if the response does not arrive as expected?
- ▶ This is why good programs need a plan for both useful responses and failed or missing responses.

19

---

---

---

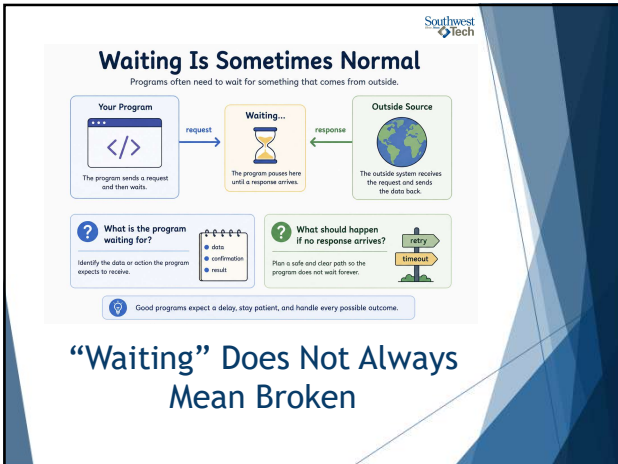
---

---

---

---

---



Southwest  
Tech

## Waiting Is Sometimes Normal

Programs often need to wait for something that comes from outside.

**Your Program**  
The program sends a request and then waits.

**request**

**Waiting...**  
The program passes here until a response arrives.

**response**

**Outside Source**  
The outside system receives the request and sends the data back.

**What is the program waiting for?**  
Identify the data or action the program expects to receive.

- data
- confirmation
- result

**What should happen if no response arrives?**  
Plan a safe and clear path so the program does not wait forever.

retry

timeout

Good programs expect a delay, stay patient, and handle every possible outcome.

## “Waiting” Does Not Always Mean Broken

20

---

---

---

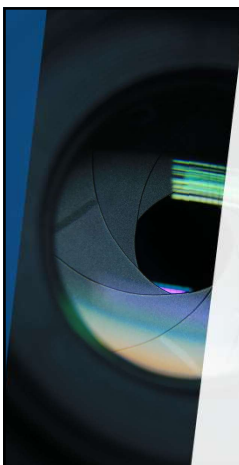
---

---

---

---

---



## Demo #1

“Request / Response Flow”

21

---

---

---

---

---

---

---

---



## What to Watch For

- ▶ where the request happens
- ▶ what response returns
- ▶ which values are selected
- ▶ what output proves the flow worked

22

---

---

---

---

---

---

---

---



## Request / Response Demo

A simple demo to see the flow in action.

- 1 request happens**  
The program asks an outside source for some data.  

- 2 response returns**  
The outside source sends data back to the program.  

- 3 values are selected**  
The program picks the values it needs.  


| Key      | Value   |
|----------|---------|
| name     | Library |
| open     | true    |
| visitors | 128     |
- 4 output proves the flow**  
The output shows the data moved through each step.  


```
Library
Open: true
Visitors: 128
```

 This demo proves the full flow: request → response → selection → output.

## Demo 1: Request/Response Flow

23

---

---

---

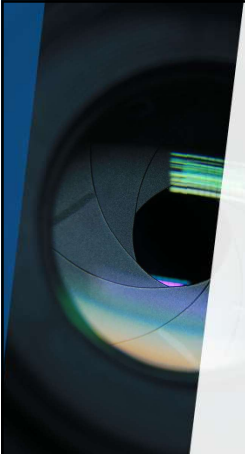
---

---

---

---

---



## Demo #2

"Async Recognition Preview"

24

---

---

---

---

---

---

---

---



## What to Watch For

- ▶ This demo is for recognition.
- ▶ You do not need to write async code today.
- ▶ You only need to recognize why waiting can change how a program is organized.
- ▶ Pay attention to the idea, not the syntax.
- ▶ The syntax can come later when the problem actually requires it.

25

---

---

---

---

---

---

---

---



## Two Ways a Program Can Move

Recognition only. Conceptual comparison.

### 1. Sequential

The program runs steps one after another.

- 1 step 1
- 2 step 2
- 3 step 3

Each step finishes before the next step begins.

### 2. Waiting-aware

The program may pause while waiting for outside data.

- 1 ask for data
- 2 wait for response  
The program pauses here. It cannot move forward yet.
- 3 continue after response  
Once data arrives, the program continues with the next step.

The program waits at a known point until the response arrives.

## Async Recognition Preview

26

---

---

---

---

---

---

---

---



## Common Failure: Vocabulary Before Meaning

Words like synchronous, asynchronous, endpoint, and response can sound bigger than the idea.

Start with the plain question: what is the program waiting for?

If you can answer that question, the vocabulary becomes easier to manage.

27

---

---

---

---

---

---

---

---

Southwest  
Tech

## Meaning Before Vocabulary

We simplify the terms to focus on the real question.

Many terms can sound complicated at first.

  
sync

  
async

  
endpoint

  
response

Useful words, but not our first priority.

Simplify



Different terms, same underlying idea.

Focus on Meaning

  
**What is the program waiting for?**

Answer this first.  
The vocabulary will make sense after the meaning is clear.

💡 When we focus on meaning, the vocabulary becomes easier to learn and remember.

## Common Failure: Vocabulary Before Meaning

28

---

---

---

---

---

---

---

---

Southwest  
Tech

## Assignment #11



- ▶ Assignment 11 will ask you to work with API-style data.
- ▶ Your first job is **not** to impress the internet.
- ▶ Your first job is to inspect the response and choose useful values.
- ▶ A small, clear response reader is better than a large program that prints raw data without understanding it.

29

---

---

---

---

---

---

---

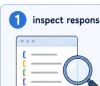
---

Southwest  
Tech

## A11 First Job

Your first task with outside data: make sense of it.

**1 inspect response**



Look at the response. What keys or sections are present?

→

**2 choose useful values**

name
☒

temp
☒

status
☒

Pick the values that answer your goal. Ignore the rest.

→

**3 explain the data path**



Briefly explain how the data moved from the response to your output.

★ Do not just dump raw JSON.  
Show only what is useful and explain how you got it.

## Assignment 11

30

---

---

---

---

---

---

---

---

## Live API And Fallback Are Both Legitimate

A live API can be useful, but it can also fail for reasons outside your code.

Simulated JSON can still teach the core skill: inspect the response, select values, and explain the flow.

**NOTE:** A common use of simulated JSON, occurs when developing an initial access for data, especially if:

- ▶ Database is not fully designed
- ▶ API's are not yet accessible
- ▶ Authorization is required but not yet available

This Photo by Unknown Author is licensed under CC BY-NC-ND

31

---

---

---

---

---

---

---

---

## Two Paths, Same Learning Target

Different sources, same skills.

**Path 1**  
Live API  
↓  
response JSON

**Path 2**  
Simulated JSON  
↓  
response JSON

Inspect, select, explain  
The same learning target for both paths.

Focus on what you do with the data, not where it came from.

## Live API & Fallback Are Both Legitimate

32

---

---

---

---

---

---

---

---

## AI Use In Lab 0n

- ▶ AI can suggest API code that looks believable but does not match the actual response.
- ▶ You must inspect the response, run the code, and explain what human decision you made.
- ▶ If AI names a field that does not exist in your response, your program is still wrong even if the code looks professional.

33

---

---

---

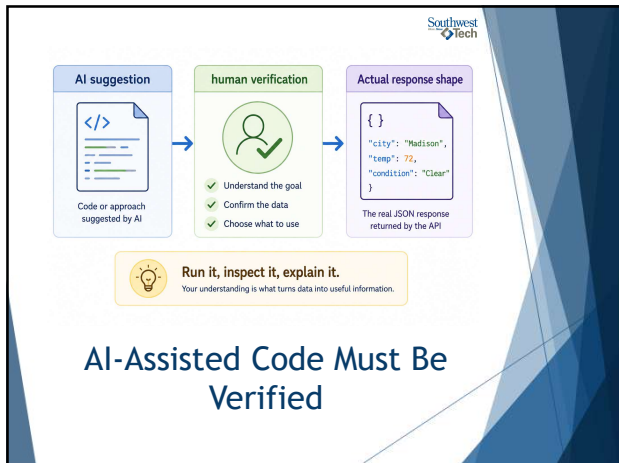
---

---

---

---

---



34

---

---

---

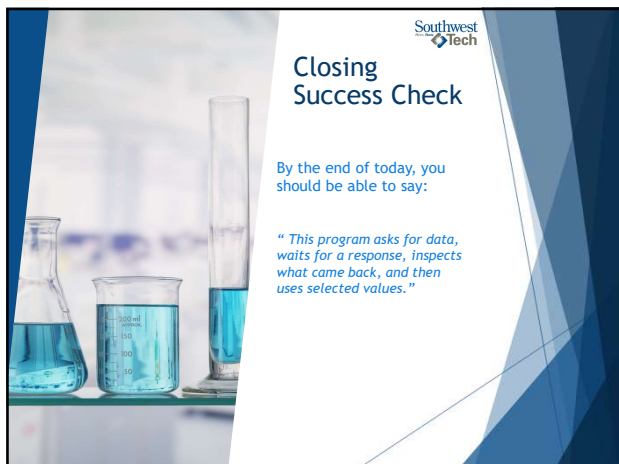
---

---

---

---

---



35

---

---

---

---

---

---

---

---



36

---

---

---

---

---

---

---

---